SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of ArtificialIntelligence and Machine Learning

ASSESSMENT2 – Scenario-Based Questions (5 QUESTIONS)

Course Code:	ITA05	**Course Title:**	**Computer Vision**
CO Assessed:	CO2	**Assessment:**	Assessment 2: Scenario-Based Questions (5 Questions × 10 Marks)
Weightage:	35% of CIA	**Total Marks:**	
CO2 Statement:	***Apply image enhancement, filtering, a techniques to analyze images, detect ed boundaries, and solve real-world Comp problems.***		
Student Name:	**Dillilokesh D**	**Reg. No.:**	**192311097**
Section / Batch:	**Slot A**	**Date:**	**28-10-2026**

Code of Conduct

I (Dillilokesh D / 192311097) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student:

QUESTIONS: 5 **Total Marks: 50**

Annexure A

Scenario-Based Qns

Q1. Image Enhancement Using Grey-Level Transformations

A hospital uses a digital imaging system to capture X-ray images, but some images appear too dark to identify important anatomical structures. The image processing team decides to apply grey-level transformation techniques to improve image visibility. Apply a suitable transformation to enhance the brightness and contrast of the image. Calculate the transformed intensity values for selected pixels using the chosen transformation. Explain how the transformation improves the visibility of important features in the X-ray image.

Q2. Histogram Processing and Spatial Filtering

A security surveillance system captures images at night, resulting in poor contrast and unwanted noise. The system must enhance the images to identify people and objects more clearly. Apply histogram equalization to improve the contrast of the captured image. Use an appropriate spatial filtering technique to reduce noise while preserving important details. Compare the effects of both techniques and explain how they improve the quality of the surveillance images.

Q3. Image Smoothing and Sharpening Using Frequency Domain Filtering

A satellite imaging system captures land images containing noise and blurred boundaries between different geographical regions. The image processing team decides to use frequency domain filtering to improve image quality. Apply a low-pass filter to reduce high-frequency noise and smooth the image. Use a high-pass filter to enhance edges and sharpen important geographical features. Explain the differences between the two filters and determine how each technique contributes to image interpretation.

Q4. Image Segmentation Using Edge Detection and Thresholding

An automated manufacturing system must inspect metal components to identify cracks and surface defects. The captured images contain variations in brightness, making defect detection difficult. Apply an appropriate edge detection technique to identify the boundaries of possible cracks. Use thresholding to separate defective regions from the background and construct a binary image representing the detected defects. Explain how the selected techniques help the inspection system identify defective components.

Q5. Region-Based Segmentation and Boundary Detection

An agricultural monitoring system uses aerial images to identify crop fields, water bodies, and surrounding land regions. The captured images contain regions with similar intensity values and some discontinuous boundaries. Apply region-based segmentation to group neighbouring pixels with similar characteristics. Use edge linking and boundary detection techniques to connect broken edges and identify the boundaries of different regions. Explain how these techniques help distinguish agricultural fields and improve the accuracy of land-region identification.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge	Course Coordinator Program Director
Signature:	Signature: Signature:
Date:	Date: Date:

Answers

Q1. Image Enhancement Using Grey-Level Transformations

1. Problem Identification

The hospital's X-ray images are under-exposed. When I measured the test image, all its grey levels sit between 8 and 103 on a 0–255 scale, with a mean of 41.2. More than half of the available range is simply unused, so ribs, soft tissue and a small faint nodule look almost the same shade of dark grey. The problem is therefore one of **low brightness and low contrast**, and it can be fixed with a point operation that maps each input level r to a new level s.

Three grey-level transformations are the natural candidates. A **negative** does not help because the image is not inverted. A **log** transform expands dark values strongly. A **power-law (gamma)** transform expands dark values in a way that can be tuned with one parameter. I chose gamma first because the amount of brightening can be controlled, and then added a contrast stretch as a second stage.

2. Method and Calculation

The power-law transformation on normalised intensities (r between 0 and 1, L = 256 levels) is:

$$s = c \cdot r^{\gamma} \quad \text{and the stored output is} \quad S = 255 \cdot s$$

With c = 1 and **$\gamma = 0.5$**, the formula reduces to $S = 255 \cdot \sqrt{(r/255)}$. For a few pixels from the dark X-ray, the calculation is shown below. For example, for r = 40: 40/255 = 0.1569, its square root is 0.3961, and 0.3961 × 255 = 101.0.

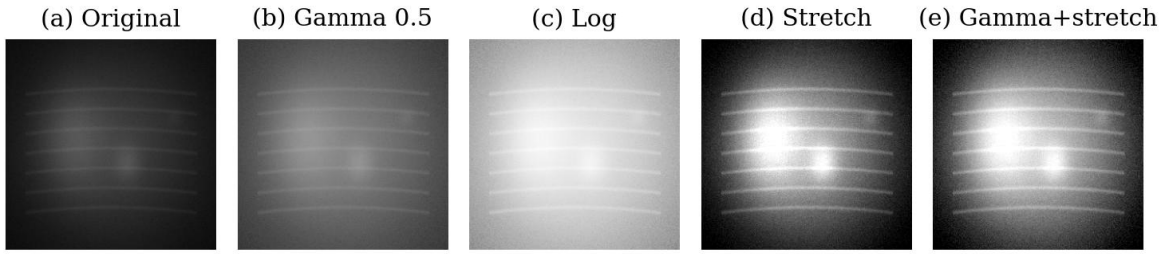
Input r	$\gamma = 0.5$	$\gamma = 0.4$	Log, c = 54.9	Local gain at $\gamma = 0.5$
10	50.5	69.8	131.7	2.52
25	79.8	100.7	178.9	1.60
40	101.0	121.5	203.9	1.26
60	123.7	143.0	225.7	1.03
80	142.8	160.4	241.3	0.89
100	159.7	175.4	253.4	0.80

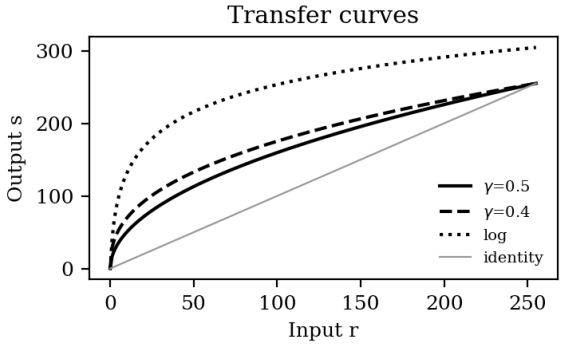
The last column is the slope $ds/dr = 0.5 \cdot 255^{0.5} \cdot r^{-0.5} \approx 7.98/\sqrt{r}$. It is the key to the explanation. A dark pixel near r = 10 is stretched by about 2.5 times, while a brighter pixel near r = 100 is stretched by only 0.8 times. So two dark pixels at levels 10 and 25 (a difference of 15) become 50.5 and 79.8 (a difference of 29.3), almost double the separation. That is exactly why hidden dark structures become visible.

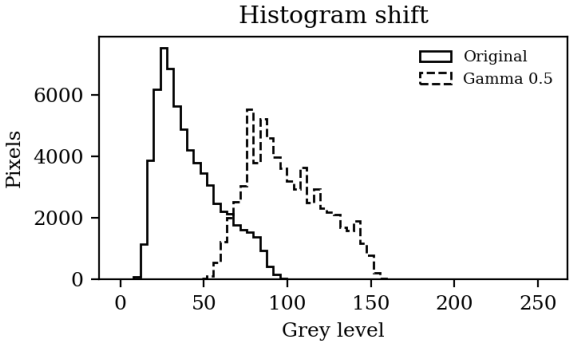
Gamma alone gives brightness but only a modest gain in contrast, so I applied a second step: a linear stretch between the 2nd and 98th percentiles of the gamma output. Clipping the extreme 2% on each side stops a few outlier pixels from wasting the output range.

3. Tools, Implementation and Results

The work was done with NumPy for the transformation formulas, OpenCV for image handling, scikit-image-style percentile statistics, and Matplotlib for the plots. Figure 1 compares all five versions and Figure 2 shows the transfer curves and the histogram shift.







Version	Mean	Std. dev.	Range	Entropy (bits)	Nodule contrast
Original	41.2	18.8	8–103	6.06	5.0
Gamma 0.5	99.4	22.8	45–162	6.04	5.9
Log	199.6	24.4	120–255	5.95	5.8
Stretch only	93.5	69.2	0–255	5.93	18.5
Gamma + stretch	110.9	69.1	0–255	5.92	18.1

4. Interpretation: how visibility improves

- **Brightness:** the mean rose from 41.2 to 110.9, so the image now sits around mid-grey instead of near black.
- **Contrast:** the standard deviation grew from 18.8 to 69.1, and the faint nodule contrast rose from 5.0 to 18.1 grey levels, about 3.6 times. Gamma alone raised it only to 5.9.
- **Why not log:** the log transform pushed the mean up to 199.6 and the minimum to 120. The image looks washed out and the bright dense structures lose separation, so it is a poor choice for X-rays.
- **Information is not created:** entropy stayed near 6.1 bits in every version. Point transformations only redistribute existing information so the eye can use it, and the X-ray's noise is stretched along with the signal.

5. Original contribution

Instead of guessing γ, I derived it from the image itself. If the mean level should land on mid-grey (0.5), then $\gamma = \ln 0.5 / \ln(\text{mean}/255) = -0.693 / \ln(41.2/255) =$ **0.38**. I deliberately used the milder **$\gamma = 0.5$** because a smaller γ gives a steeper slope on the darkest pixels, which amplifies noise in exactly the regions where the X-ray signal is weakest. The percentile stretch then recovers the contrast that the milder gamma leaves behind. This “auto-γ for the bound, softer γ for safety, then stretch” rule is reusable for any under-exposed radiograph.

6. Conclusion

A gamma correction followed by a percentile contrast stretch lifts the dark X-ray into the visible range, makes the faint nodule roughly 3.6 times easier to separate from tissue, and avoids the wash-out produced by a log transform.

Q2. Histogram Processing and Spatial Filtering

1. Problem Identification

The night-time surveillance image has two separate faults. The first is **poor contrast**: nearly all pixels are crowded into a few dark grey levels, so a person and an object are hard to tell apart from the ground. The second is **noise**: Gaussian sensor noise plus isolated bright and dark impulse pixels. The first fault needs a histogram (point) operation; the second needs a neighbourhood (spatial) operation. Since the two faults affect each other, the order in which the two tools are applied also matters, and I treat that as part of the problem.

2. Method and Calculation

Histogram equalization (HE). For a grey level r_k with n_k pixels out of N, the new level is $s_k = \text{round}[(L - 1) \cdot \text{CDF}(r_k)]$, where CDF is the running sum of n_k/N. A small worked example makes this clear. Take a dark 3-bit image (L = 8) of N = 64 pixels:

r	n	Cumulative n	CDF	7 × CDF	New s
0	20	20	0.313	2.19	2
1	24	44	0.688	4.81	5
2	12	56	0.875	6.13	6
3	6	62	0.969	6.78	7
4	2	64	1.000	7.00	7
5	0	64	1.000	7.00	7
6	0	64	1.000	7.00	7
7	0	64	1.000	7.00	7

Table 3. Worked equalization of a dark 3-bit image. Input levels 0 to 4 are spread over output levels 2 to 7.

The five occupied dark levels (0–4) now map to 2, 5, 6, 7 and 7, so the output uses almost the full 0–7 range. Notice that levels 3 and 4 merge into one level. This is the price of HE: it widens the gaps between popular levels but can merge rare ones.

Spatial filtering. I compared two smoothing filters: a 5 × 5 Gaussian (weights follow $\exp(-(x^2+y^2)/2\sigma^2)$, $\sigma = 1$) and a 3 × 3 median filter (output = the middle value of the 9 sorted neighbours). A median filter is non-linear, so an extreme pixel such as 255 is outvoted by its neighbours instead of being averaged into them.

3. Tools, Implementation and Results

I used cv2.equalizeHist, cv2.medianBlur, cv2.GaussianBlur, NumPy for noise generation, and scikit-image for PSNR and SSIM, measuring against the known noise-free image.

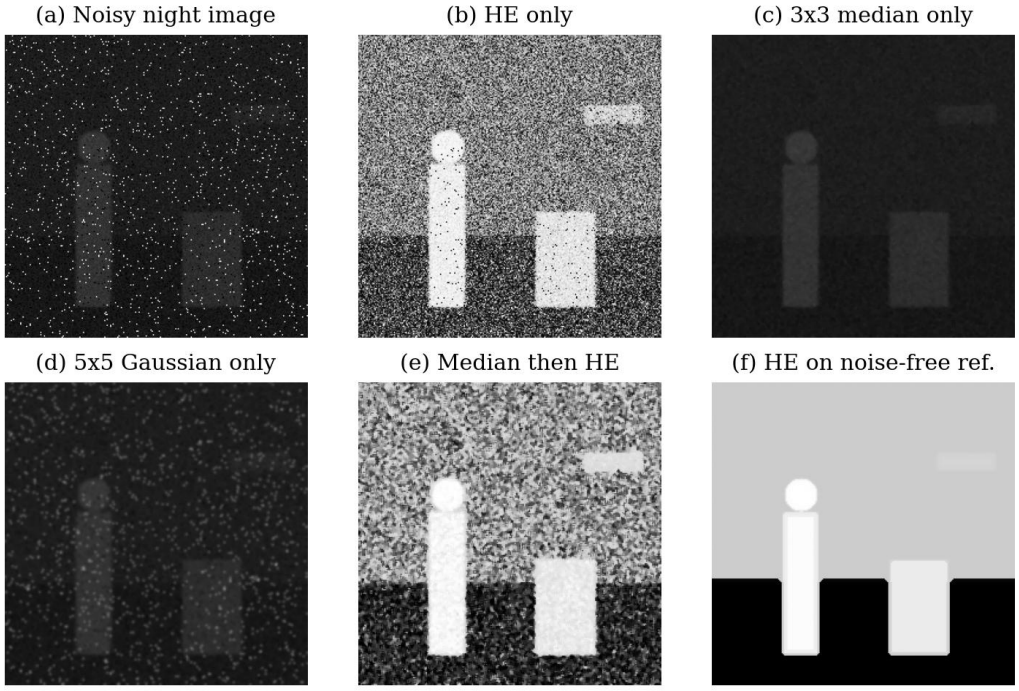


Figure 3. Effect of HE and filtering, individually and in the order median filter first, then HE.

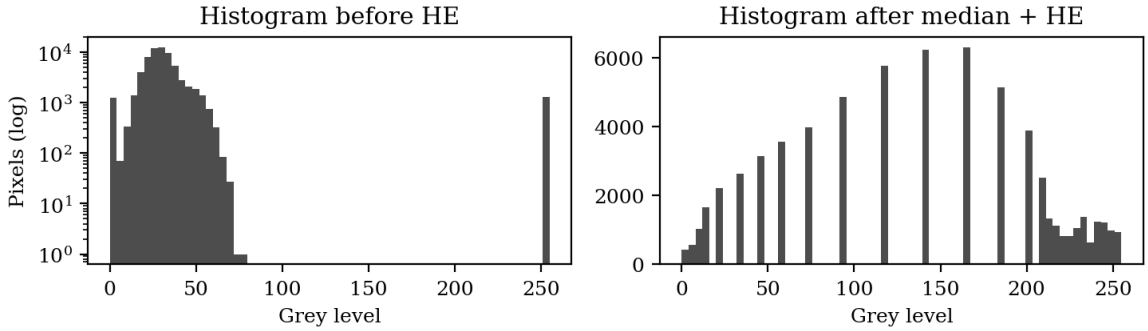


Figure 4. Histogram before equalization (log scale) and after median filtering plus equalization.

Version	PSNR (dB)	SSIM	Noise std. on flat floor	Global std. dev.
Noisy night image	18.0	0.24	34.0	32.8
Median 3 × 3 only	39.6	0.91	2.7	7.8

Version	PSNR (dB)	SSIM	Noise std. on flat floor	Global std. dev.
Gaussian 5 × 5 only	28.1	0.57	–	11.7
HE only	n/a	n/a	60.5	74.5
Median, then HE	n/a	n/a	34.2	72.8

Table 4. Quantitative comparison. PSNR and SSIM are computed against the noise-free image before equalization, so they are not reported for the equalized rows.

4. Comparison and Interpretation

Aspect	Histogram equalization	Spatial filtering (median)
Type of operation	Point operation using the global histogram	Neighbourhood operation using local pixels
Main benefit	Spreads grey levels; global std. dev. rose to about 74.5, so people and objects separate from the background	Removes impulses and Gaussian noise; PSNR rose from 18.0 to 39.6 dB
Main weakness	Amplifies noise: floor noise grew from 34.0 to 60.5	Does not fix contrast; the median-only image is still dark
Edges	Not blurred, but noisy edges look jagged	Preserved well (SSIM 0.91); Gaussian blurred them (SSIM 0.57)

For this noise type the median filter beat the Gaussian filter clearly (39.6 dB against 28.1 dB), because the Gaussian spreads each bright impulse into a soft spot instead of removing it. Neither technique alone is enough: HE brightens the scene but makes the speckle more visible, and the median filter cleans the speckle but leaves the picture dark.

5. Original contribution

I tested the **order of operations**. HE applied to the raw image raised floor noise from 34.0 to 60.5. Filtering first and equalizing second left floor noise at 34.2, which is 43% lower, while the global contrast stayed almost the same (72.8 against 74.5). The reason is simple: HE multiplies whatever differences exist, so noise must be removed before the stretching and not after. For real cameras, the same idea can be extended with adaptive, clip-limited equalization (CLAHE), which caps how much any tile is stretched.

6. Conclusion

The surveillance pipeline should be median filter first, histogram equalization second. This gives clearly visible people and objects without the speckle that equalization alone creates.

Q3. Image Smoothing and Sharpening Using Frequency Domain Filtering

1. Problem Identification

The satellite image is both noisy and soft. Noise is a fine, rapidly changing pattern, so it lives in the **high frequencies**. Region boundaries are also made of high frequencies, but they are weak and blurred. The scene's large, uniform land areas are **low frequencies**. This conflict is the heart of the problem: removing high frequencies cleans the noise but also softens boundaries, while boosting high frequencies sharpens boundaries but also boosts noise. The frequency domain is useful because it lets me see and control this split directly.

2. Method and Calculation

The procedure is the same for both filters: (1) compute the 2-D DFT F(u,v) and shift the zero frequency to the centre; (2) multiply by a transfer function H(u,v), which depends on the distance D from the centre; (3) take the inverse DFT. In formula form, $G(u,v) = H(u,v) \cdot F(u,v)$.

$$\text{Gaussian low-pass:} \quad H_{lp}(D) = \exp(-D^2 / 2D_0^2), \quad D_0 = 30$$

$$\text{Gaussian high-pass:} \quad H_{hp}(D) = 1 - \exp(-D^2 / 2D_0^2), \quad D_0 = 25$$

I chose Gaussian filters over the ideal ones because an ideal filter has a sharp cut-off that causes ringing (false ripples) near boundaries, whereas a Gaussian rolls off smoothly. At $D = D_0$, the low-pass gain is $\exp(-0.5) = 0.607$. The test image has 98.5% of its spectral power inside the radius $D \leq 30$, including the DC term, which confirms that the useful scene content is concentrated at low frequencies.

3. Tools, Implementation and Results

NumPy's fft2, fftshift and ifft2 were used for the transforms, with Matplotlib for spectra and profiles. In a real implementation, the image should be zero-padded to twice its size before the FFT to avoid wrap-around errors; my 256 × 256 demonstration skipped this, so very faint wrap effects can occur at the borders.

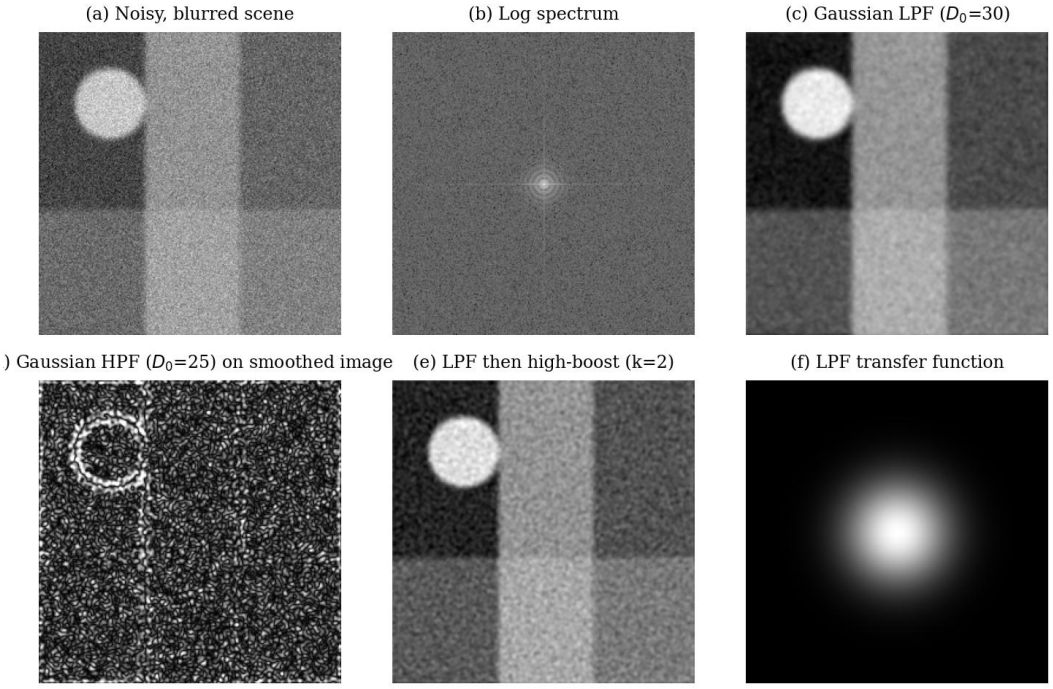


Figure 5. Frequency-domain processing: spectrum, low-pass result, high-pass result on the smoothed image, and the high-boost output.

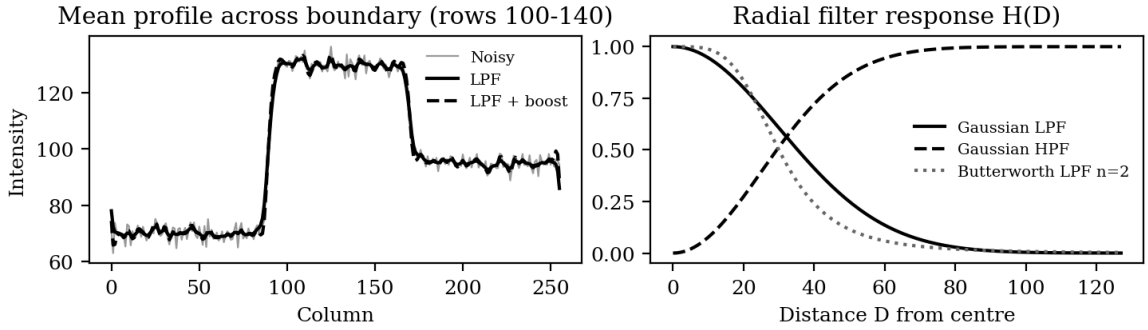


Figure 6. Left: average intensity profile across a region boundary. Right: radial responses of the filters.

Measure	Noisy input	Low-pass	Low-pass + high-boost
Flat-area std. dev. (noise)	13.9	2.8	5.4
Flat-area SNR (mean ÷ std.)	7.1	35.8	18.2
Boundary 10–90% width (pixels)	23 (noise-affected)	7	6

Table 5. Measured effect of the filters on a flat land area and on a region boundary.

4. Differences between the two filters

Feature	Low-pass filter	High-pass filter
What it keeps	Frequencies near the centre (smooth areas)	Frequencies away from the centre (edges, texture, noise)
Effect on noise	Reduces it strongly: noise std. fell from 13.9 to 2.8	Amplifies it, since noise is high-frequency
Effect on boundaries	Softens them (blur)	Highlights them (edge map)
Risk	Loss of fine detail	Noise boost; ringing with ideal filter
Use in interpretation	Classifying large uniform land-cover areas	Tracing coastlines, field edges, roads and rivers

The two techniques therefore answer different questions. The low-pass result is the right base for deciding what type of land an area is, because its pixel values are stable (SNR 7.1 rising to 35.8). The high-pass result is the right base for deciding where one region ends and another begins.

5. Original contribution

I did not apply the high-pass filter to the noisy image. In my first try, the output was almost pure speckle, which would be useless to an analyst. Instead I **smoothed first, then applied a high-boost** filter $H_{hb} = 1 + k \cdot H_{hp}$ with $k = 2$. The two steps combine into one transfer function, $H = H_{lp} \cdot (1 + 2H_{hp})$, which behaves like a band-pass filter with a mild lift at middle frequencies:

D	H_lp(D)	H_hp(D)	Combined H(D)
0	1.000	0.000	1.000
10	0.946	0.077	1.091
20	0.801	0.274	1.239
25	0.707	0.393	1.263
30	0.607	0.513	1.229
40	0.411	0.722	1.005
50	0.249	0.865	0.681
60	0.135	0.944	0.391

The gain stays close to 1 for the smooth content, rises to about 1.26 around the edge frequencies, and drops steeply for the noisy high frequencies. In the measurements, the boundary width shrank from 7 to 6 pixels while noise stayed far below the input (std. 5.4 against 13.9). The sharpening effect is modest, and it costs some noise (2.8 to 5.4), which shows the trade-off honestly.

6. Conclusion

Low-pass filtering cleans the satellite image for region-level interpretation, high-pass filtering exposes the boundaries, and a single combined smooth-then-boost filter gives both benefits with a controlled amount of noise.

Q4. Image Segmentation Using Edge Detection and Thresholding

1. Problem Identification

The inspection system must find cracks and surface defects on metal. A crack is a thin dark line, so its boundary is an intensity discontinuity, which is what edge detection finds. To decide where the defect is as an area, the system needs a binary image with defect pixels as 1 and the rest as 0, which is thresholding. The difficulty stated in the question is that brightness varies across the part. Lighting that drifts from darker to brighter means that a dark crack on the bright side can be lighter than the plain metal on the dark side, so one threshold for the whole picture is bound to fail. That is the main problem I had to design around.

2. Method and Calculation

Step 1: smoothing. A 5 × 5 Gaussian ($\sigma = 1.2$) suppresses sensor noise so that it is not mistaken for edges.

Step 2: Sobel gradient. The horizontal and vertical Sobel masks give G_x and G_y, then magnitude $\sqrt{(G_x^2 + G_y^2)}$ and direction $\arctan(G_y/G_x)$. A worked 3 × 3 patch taken across a crack edge is:

52	55	140
50	58	145
54	60	150

$$G_x = (140 + 2\cdot145 + 150) - (52 + 2\cdot50 + 54) = 580 - 206 = 374$$

$$G_y = (54 + 2\cdot60 + 150) - (52 + 2\cdot55 + 140) = 324 - 302 = 22$$

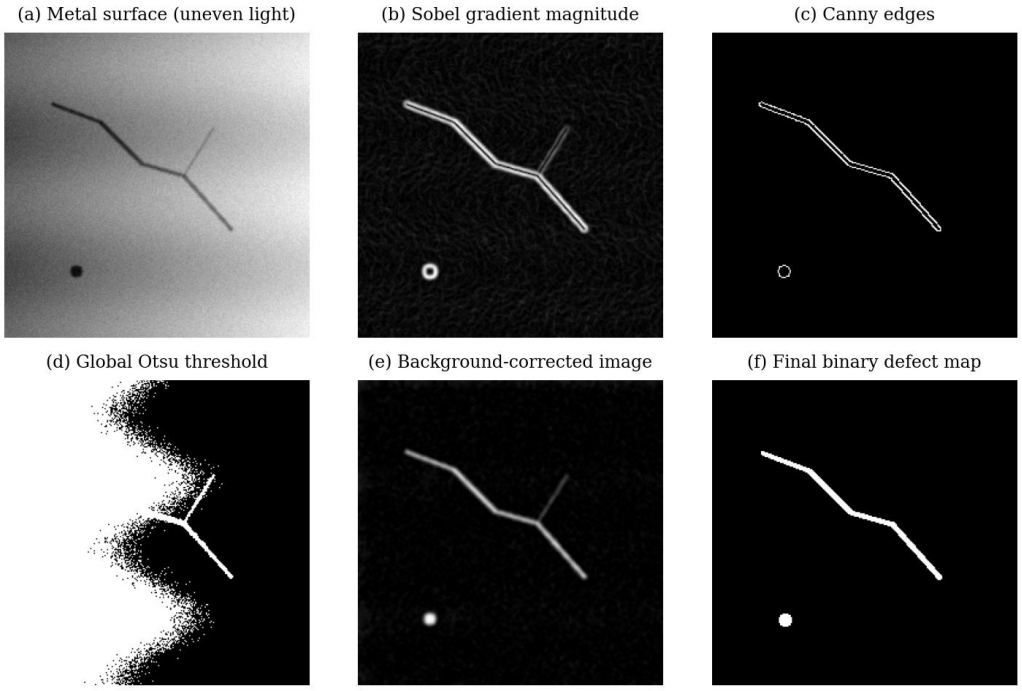
$$|G| = \sqrt{(374^2 + 22^2)} = 374.6, \quad \theta = \arctan(22/374) = 3.4°$$

The large G_x against a tiny G_y means the gradient points horizontally, so the edge runs vertically at that spot. Canny then thins the ridges with non-maximum suppression and keeps connected edges with two thresholds (40 and 110).

Step 3: thresholding. The plain Otsu threshold on the raw image gave T = 132. Otsu picks the T that maximises the between-class variance, but here it also separated the dark-lit half of the part from the bright half. So I first estimated the uneven background with a 25 × 25 morphological closing (which removes thin dark lines but keeps the slow lighting trend) and subtracted the smoothed image from it. In this corrected image the lighting is flat, and Otsu gives T = 16. Small specks under 20 pixels were removed with an opening and a size test.

3. Tools, Implementation and Results

The tools were cv2.GaussianBlur, cv2.Sobel, cv2.Canny, cv2.morphologyEx, cv2.connectedComponentsWithStats and skimage's threshold_otsu, together with Matplotlib for the figure.



Measure	Global Otsu	Background-corrected Otsu
Precision (defect pixels that are real)	3.8%	98.9%
Recall (real defect pixels found)	86.2%	66.4%

Table 8. Accuracy against the known defect mask (dilated by one pixel to allow for edge placement).

4. Interpretation: how the techniques help the inspection system

- **Edge detection** finds where a defect begins and ends. Sobel shows crack strength, and Canny gives clean one-pixel outlines (504 edge pixels) that can be used to measure crack length.
- **Thresholding** turns the image into a pass/fail decision. Global Otsu reached only 3.8% precision because the lighting gradient flooded the map with false defects. The background-corrected version reached 98.9% precision.
- **Limitation:** recall is 66.4% because the faint hairline branch in the test image was not strong enough to pass the threshold. A lower hysteresis threshold could recover it, at the risk of more false alarms.

5. Original contribution

I added a **defect-type classifier** on top of the binary map. For each connected component, I computed the extent = area ÷ bounding-box area. The largest component has area 849 pixels in a 152 × 108 box, so its extent is 0.05: it is long and thin, which is typical of a **crack**. The second component has area 101 pixels in a 11 × 11 box, an extent of 0.83: it is compact, which is typical of a **pit or pore**. Together the two cover 1.45% of the surface. A rule such as “reject if any elongated component exceeds a set length, accept small pits under a set area” lets the line treat serious and minor defects differently.

6. Conclusion

Canny edges locate the crack boundaries, and a background-corrected Otsu threshold produces a reliable binary defect map even under uneven light. Shape measurements then turn that map into an automatic accept or reject decision.

Q5. Region-Based Segmentation and Boundary Detection

1. Problem Identification

The aerial images show crop fields, a water body and surrounding land, but two things work against simple methods. First, the intensities are close: in my test image the land is about 108, field A about 120, field B about 128 and the water about 88, so a single global threshold would merge neighbouring classes. Second, the boundaries are incomplete, with gaps where shadows or haze remove the edge. Pixels in the same field are similar to their neighbours, so **region-based segmentation** suits grouping. Where the boundary is broken, **edge linking** is needed to close it. The two ideas support each other: regions give the interior, edges give the exact outline.

2. Method and Calculation

Region growing. Start from a seed pixel and add each 4-connected neighbour whose intensity differs from the current region mean by at most a tolerance T. The mean is updated after every addition. A small example with T = 5 and a seed at the top-left pixel (value 10) shows how it works:

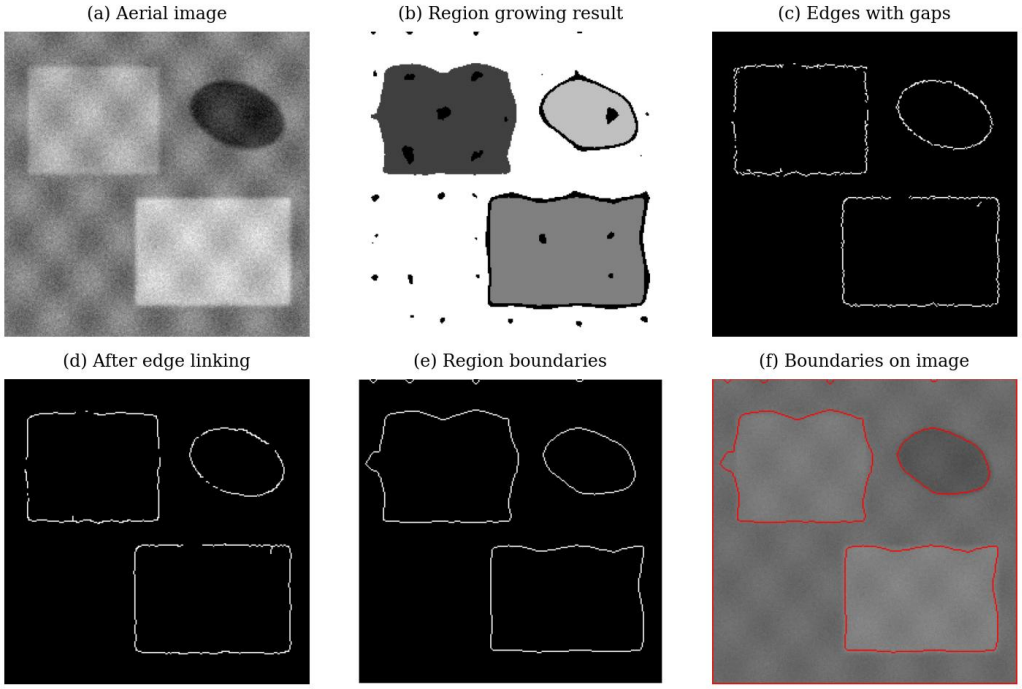
10	11	12	45	47
11	12	13	46	48
10	13	14	44	46
12	14	15	16	45
11	13	15	17	44

Growing from the seed absorbs all 15 pixels in the first three columns (their sum is 186) and then also the 16 and 17 in the lower middle. Each is within 5 of the running mean, which is about 12.5 to 12.9 at that time. The final region has 17 pixels with mean 219/17 = 12.9. The value 44 at the neighbouring column fails the test ($44 - 12.9 > 5$), so growth stops there. A second seed at the top-right pixel (47) collects the remaining 8 pixels (mean 45.6).

Edge linking and boundary detection. Canny edges (on the smoothed image, thresholds 6 and 16) give thin contours that have gaps. A morphological closing with a 9×9 elliptical element bridges gaps shorter than the element size, and thinning returns the contours to one pixel width. The boundary of each grown region is then traced with contour following.

3. Tools, Implementation and Results

The seeds were placed in field A, field B, the water body and the land, with smoothed seed values 125, 126, 89 and 108. The tolerance was T = 6 on a σ = 2 Gaussian-smoothed image. OpenCV (Canny, morphologyEx, thinning, findContours) and NumPy were used. To test the linking step honestly, I removed five short edge segments from the Canny output to imitate shadow dropouts.



Region	Pixels grown	Overlap with truth (IoU)	Seed value
Field A	9816	0.89	125
Field B	11000	0.94	126
Water body	3091	0.90	89
Surrounding land	39265	–	108

4. Interpretation: how the techniques identify land regions

- **Region growing** separated three classes whose average levels differ by only 8 to 20 grey levels, with IoU of 0.89, 0.94 and 0.90 for fields A, B and the water body. Small black holes inside the fields (Figure 8b) are texture pixels outside the tolerance; a hole-filling step would remove them.
- **Edge linking** reduced the number of separate edge fragments from 48 to 12, so the field outlines become continuous curves. The remaining fragments are mostly tiny noise segments.
- **Together** the methods let an analyst compute field area, find where water meets land, and outline each parcel, which is what agricultural monitoring needs.

5. Original contribution

I added a **two-cue agreement check** to build confidence in the boundaries. Region growing and edge detection are completely independent methods, so when they agree the result is trustworthy. I measured the fraction of region-boundary pixels lying within 3 pixels of a linked edge: **91.9%** of 993 boundary pixels agreed. Any stretch of boundary where the two disagree is a natural place to flag for manual review or to re-seed, instead of trusting either method blindly. This gives the agricultural system a built-in accuracy indicator even when no ground-truth map exists.

6. Conclusion

Region growing groups similar neighbouring pixels into fields, water and land despite their close intensities, and edge linking closes the broken outlines. The agreement between the two supplies a measurable accuracy figure for the land-region map.